Faculdade de Tecnologia de Sorocaba
Tecnologia em Análise e Desenvolvimento de Sistemas

KARINA DE MORAES GARCIA
NICOLLY LEMOS DA SILVA
RAFAELA MANSANO FERNANDES
RAFAELA SILVA RIGANTI

PADRÕES DE PROJETO NO DRE SYSTEM

Trabalho da disciplina de Padrões de Projeto, apresentado à Profª Cristiane Palomar, sobre os padrões utilizados no sistema do Projeto Integrador II.

SOROCABA/SP
2026

1. Introdução

O **DRE System** (OAF Controle Financeiro) é o sistema do nosso Projeto Integrador II: um controle financeiro web feito em PHP 8 e MySQL, sem framework, que gera o DRE (Demonstrativo do Resultado do Exercício), o fluxo de caixa, o calendário de contas a pagar e a receber e guarda as notas fiscais ligadas aos lançamentos.

Neste trabalho apresentamos os padrões de projeto usados no programa. Três deles são do catálogo GoF e foram aplicados onde o código tinha `if/else` crescendo ou classes que sabiam demais umas das outras: **Strategy**, **Factory** e **Observer**. Os outros são padrões de arquitetura que já davam forma ao sistema desde o começo: **MVC**, **Front Controller**, **Singleton**, **Facade** e **Layer Supertype**. No fim está o diagrama de classes com todos eles.

A aplicação dos padrões não mudou nenhuma funcionalidade. Rodamos os mesmos relatórios e operações antes e depois da refatoração e os resultados foram idênticos.

2. Padrões GoF aplicados

Repositório: github.com/nicollylemos/DRE

2.1 Strategy — Regime e agrupamento do DRE

Categoria	• Padrão comportamental (GoF).
Problema	• A tela do DRE pode ser vista em dois regimes (competência ou caixa) e agrupada de três jeitos (categoria, grupo/conta ou centro de custo). • Antes, o `DreService` decidia isso com vários `if` no meio do cálculo, e a lista de opções ficava repetida na exportação em PDF.
Solução	• Cada regime virou uma classe que implementa a interface `RegimeDre` (`RegimeCompetencia`, `RegimeCaixa`). • Cada agrupamento virou uma classe que implementa `AgrupamentoDre` (`AgrupamentoPorCategoria`, `AgrupamentoPorGrupoConta`, `AgrupamentoPorCentroCusto`). • O `DreService` é o contexto: recebe as estratégias escolhidas pela classe `EstrategiasDre` e monta o demonstrativo sem saber qual está usando.
Onde está	• `app/Services/Dre/` e `app/Services/DreService.php`.
Benefícios	• Um agrupamento novo (por fornecedor, por exemplo) é uma classe pequena registrada em uma linha, sem mexer no cálculo. • Os nomes exibidos na tela e no PDF vêm das próprias estratégias, num lugar só.

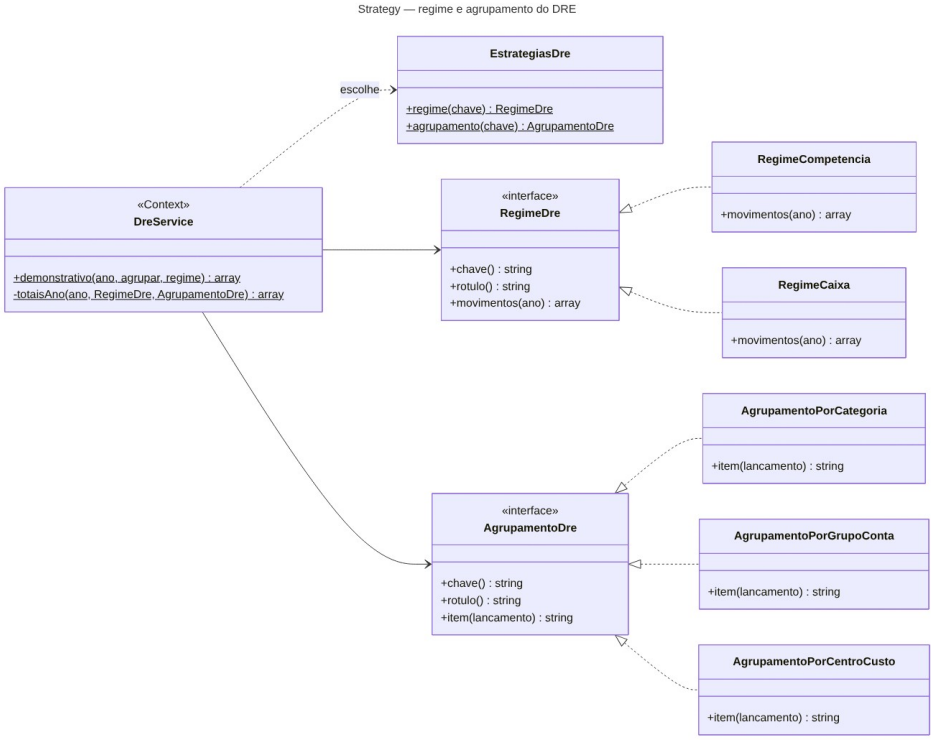


Figura 1 — Strategy no cálculo do DRE

2.2 Factory — Leitura do XML da nota fiscal

Categoria	• Padrão criacional (GoF), no estilo Factory Method: a fábrica decide qual subclasse de leitor criar.
Problema	• O sistema lê três layouts de nota: NF-e/NFC-e (SEFAZ), NFS-e Nacional e NFS-e ABRASF das prefeituras. • Tudo ficava numa classe só, com uma sequência de `if` decidindo o layout e um método enorme para cada um.
Solução	• Cada layout virou um leitor, subclasse de `LeitorNotaFiscal` (`LeitorNfe`, `LeitorNfseNacional`, `LeitorNfseAbrasf`), que sabe reconhecer o próprio XML e extrair os campos. • A `FabricaLeitorNotaFiscal` recebe o XML carregado (`DocumentoXml`), pergunta aos leitores qual o reconhece e devolve o objeto certo. • Quem usa (cadastro de nota, importação em lote, sugestão de lançamento) só chama `ler()` e não conhece nenhum layout.
Onde está	• `app/Services/NotaFiscal/` e `app/Services/NotaFiscalXml.php`.
Benefícios	• Um layout novo de prefeitura é só uma subclasse a mais incluída na fábrica. • O que é comum (datas, valores, CNPJ e formato da resposta) ficou na classe base, sem repetição, e a segurança contra XML malicioso ficou isolada no `DocumentoXml`.

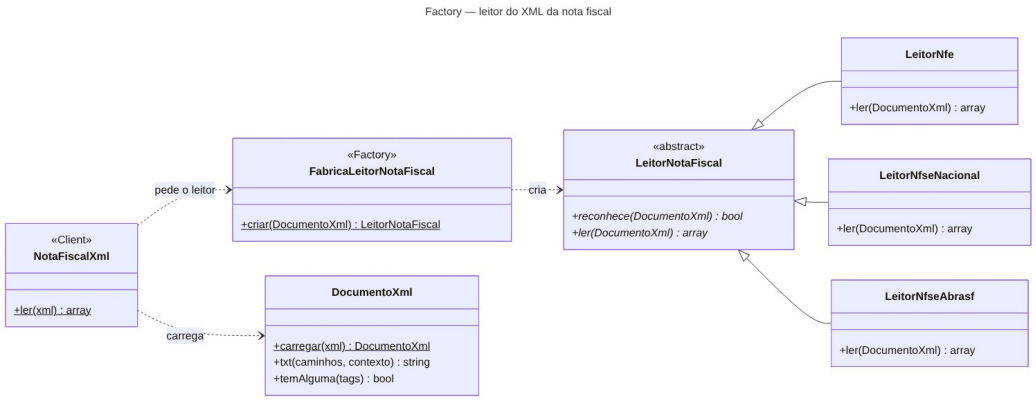


Figura 2 — Factory na leitura das notas fiscais

2.3 Observer — Eventos dos lançamentos

Categoria	• Padrão comportamental (GoF).
Problema	• O controller de lançamentos precisava conhecer regras de outras partes: ao criar um lançamento a partir de uma nota, vincular os dois; ao apagar, tirar o vínculo da nota e apagar o comprovante anexado. • Cada regra nova desse tipo deixava o controller mais acoplado.
Solução	• O `TransacaoController` só avisa que algo aconteceu, disparando um `Evento` (`lancamento.criado` ou `lancamento.apagado`) no `Despachante`, que é o sujeito observado. • Quem precisa reagir implementa a interface `Ouvinte` e é registrado em `app/eventos.php`: `VincularNotaFiscal`, `DesvincularNotasFiscais` e `ApagarComprovante`.
Onde está	• `app/Eventos/`, `app/eventos.php` e `app/Controllers/Api/TransacaoController.php`.
Benefícios	• O controller não conhece mais notas fiscais nem arquivos de comprovante. • O padrão provou o valor na prática: quando entrou o comprovante de pagamento, apagar o arquivo junto com o lançamento virou só mais um ouvinte, sem tocar no controller.

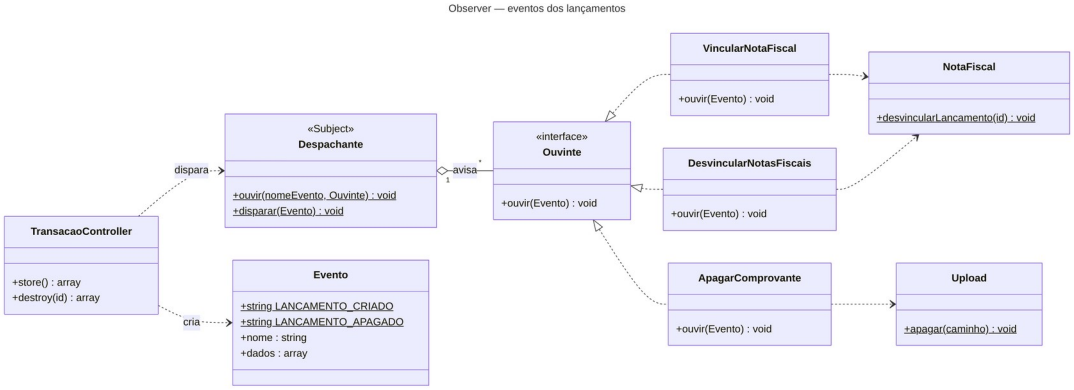


Figura 3 — Observer nos eventos de lançamento

3. Padrões de arquitetura do sistema

Além dos três padrões GoF, o sistema já era organizado com padrões de arquitetura que aparecem no diagrama de classes.

3.1 MVC — Model, View e Controller

Como funciona	• **Models** (`app/Models`) concentram o SQL; **Views** (`app/Views`) montam o HTML; **Controllers** (`app/Controllers`) recebem a requisição, conferem a permissão e respondem.
Benefícios	• Cada camada muda sem quebrar as outras: dá para trocar o layout de uma tela sem mexer na consulta, e vice-versa.

3.2 Front Controller — Ponto de entrada único

Como funciona	• Toda requisição entra por `index.php`, que chama `App::run()`. • O `App` aplica as travas globais (CSRF, troca de senha obrigatória, aceite da LGPD) e usa o `Router` para chamar o controller certo.
Benefícios	• Segurança e regras gerais ficam num lugar só, e nenhuma tela consegue ser acessada sem passar por elas.

3.3 Singleton — Conexão com o banco

Categoria	• Padrão criacional (GoF).
Como funciona	• A classe `Database` guarda uma única conexão PDO por requisição e todo o sistema a reaproveita por `Database::pdo()`.
Benefícios	• Evita abrir várias conexões no mesmo pedido e garante que transações (como a importação em lote) usem a mesma conexão do começo ao fim.

3.4 Facade — Leitura de notas fiscais

Categoria	• Padrão estrutural (GoF).
Como funciona	• A classe `NotaFiscalXml` oferece um único método, `ler($xml)`, e esconde por trás dele o `DocumentoXml`, a fábrica e os leitores.
Benefícios	• O resto do sistema continuou chamando a mesma função de antes da refatoração, sem precisar conhecer as classes novas.

3.5 Layer Supertype — Classes base

Como funciona	• `Model` e `Controller` são classes abstratas que concentram o que todos os filhos usam: CRUD básico e nome da tabela no `Model`; autorização, leitura da requisição e registro de log no `Controller`.
Benefícios	• Um model novo herda o CRUD pronto e um controller novo já nasce com a checagem de permissão e o log de auditoria.

4. Diagrama de classes

O diagrama abaixo reúne todos os padrões do sistema, agrupados por pacote: Front Controller/MVC, classes base, Singleton, controllers, models, Strategy, Factory e Observer. Os pacotes com borda dourada são os padrões GoF. O arquivo editável no draw.io está no repositório em `docs/padroes/diagrama-classes.drawio`.

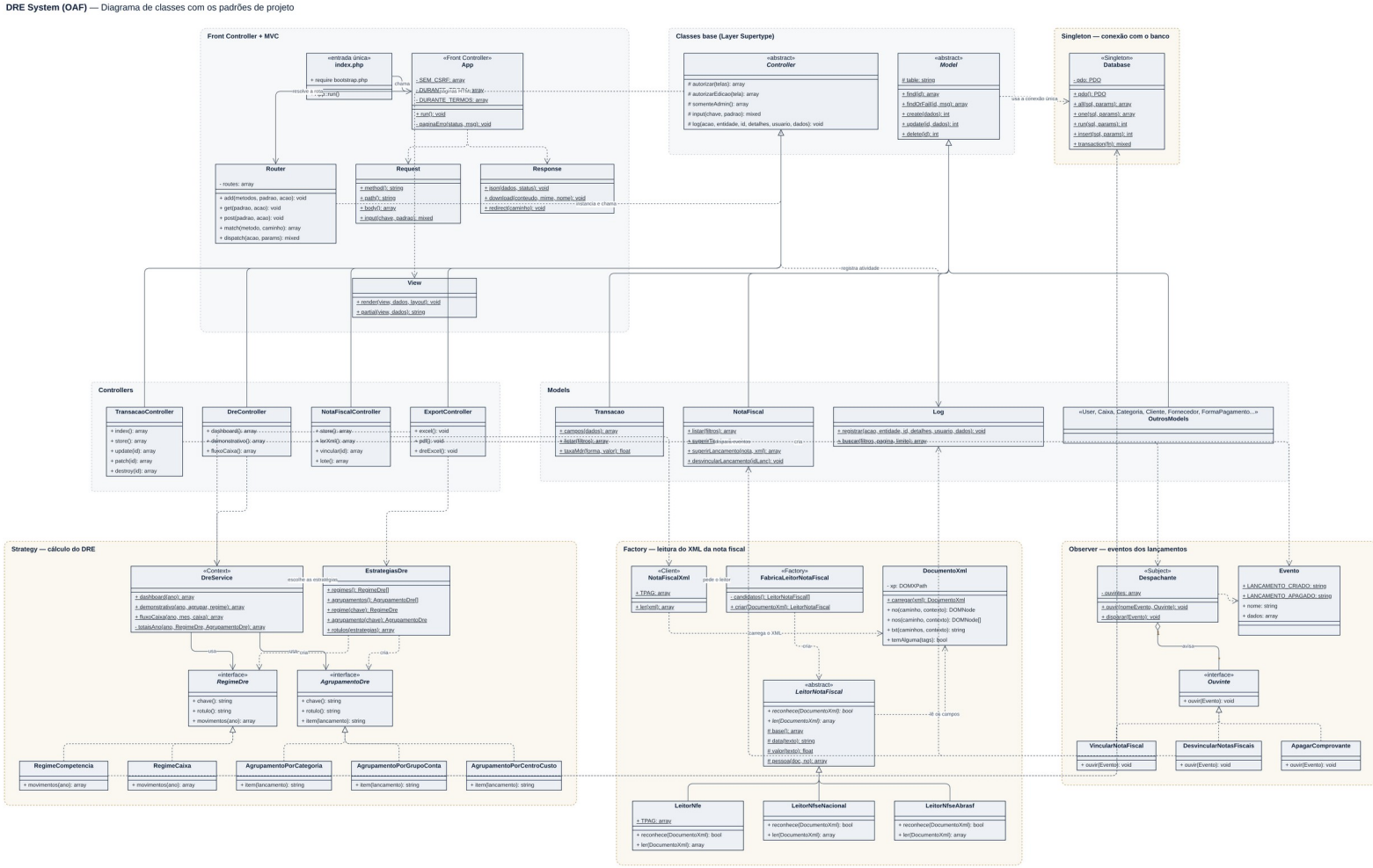


Figura 4 — Diagrama de classes do DRE System com os padrões de projeto (draw.io)

5. Como conferimos que nada mudou

Antes da refatoração geramos os dados fictícios do sistema (cerca de 2.000 lançamentos e 47 notas com XML) e salvamos o resultado de todos os relatórios: dashboard, DRE nos dois regimes e nos três agrupamentos, fluxo de caixa de 24 meses e a leitura de todos os XML, inclusive os inválidos. Depois da refatoração rodamos tudo de novo e o resultado foi idêntico byte a byte. Também repetimos pela API o mesmo roteiro nas duas versões (criar nota pelo XML, gerar o lançamento vinculado, apagar, importar em lote e exportar) e as respostas foram as mesmas.

6. Conclusão

Os padrões entraram onde o código mais sofria com mudanças: o cálculo do DRE, a leitura das notas e as reações aos lançamentos. Com eles, cada novidade passou a ser uma classe nova em vez de mais um `if` em código que já funcionava. O melhor exemplo veio logo depois: o comprovante de pagamento precisou ser apagado junto com o lançamento, e bastou criar um ouvinte, sem mexer no controller.